

From Vibe Coding to Spec-Driven Agentic Engineering

A Hands-On Implementation using Claude Code

PART VI

Instructor: Muhammad Arif Butt, PhD

About The Instructor



Dr. Muhammad Arif Butt
Asst. Prof. Department of Data Science
University of Punjab, Lahore
arif@pucit.edu.pk
https://arif.excaliat@gmail.com
https://arifbutt.me
https://www.linkedin.com/in/dr-arif-butt/
https://youtube.com/learnwitharif
https://github.com/arifpucit
https://huggingface.co/arif-butt

- **Education:**
  - Graduation from Pakistan Military Academy, Kakul
  - MPhil CS from University of Punjab, Lahore
  - PhD CS from University of Punjab, Lahore
- **Experience:**
  - Served in field/staff/instructional posts in Pakistan Army
  - Assistant Professor, Department of Data Science
  - Founder Excaliat (https://excaliat.com/en)
  - Founder FalconHunt (https://falconhunt.org/)
  - Co-Founder Tbox Solutionz (https://tboxsolutionz.com/)
- **Research Interest:**
  - Embedded and Real-Time Operating Systems
  - Vulnerability Analysis, Binary Exploitation & Exploit Development
  - AI-Driven Cybersecurity and Securing AI systems

Scheme of Sessions

Part - I: Overview of Agentic Coding Assistants

Part – II: Claude Ecosystem, Installing & using Claude Code

Part – III: Slash Commands, Context Window & `.claude` Directory

Part – IV: Workflow of Building a Basic Vulnerable Web Application

Part – V: Exploiting Vulnerable Web Application

Part – VI: Workflow of Implementing Features and Fixing Bugs

Part – VII: Skill Marketplace and Writing your Own Skills

Part – VIII: Creating and using Subagents & Agent Teams

Part – IX: Model Context Protocol

Part – X: Creating and using Hooks

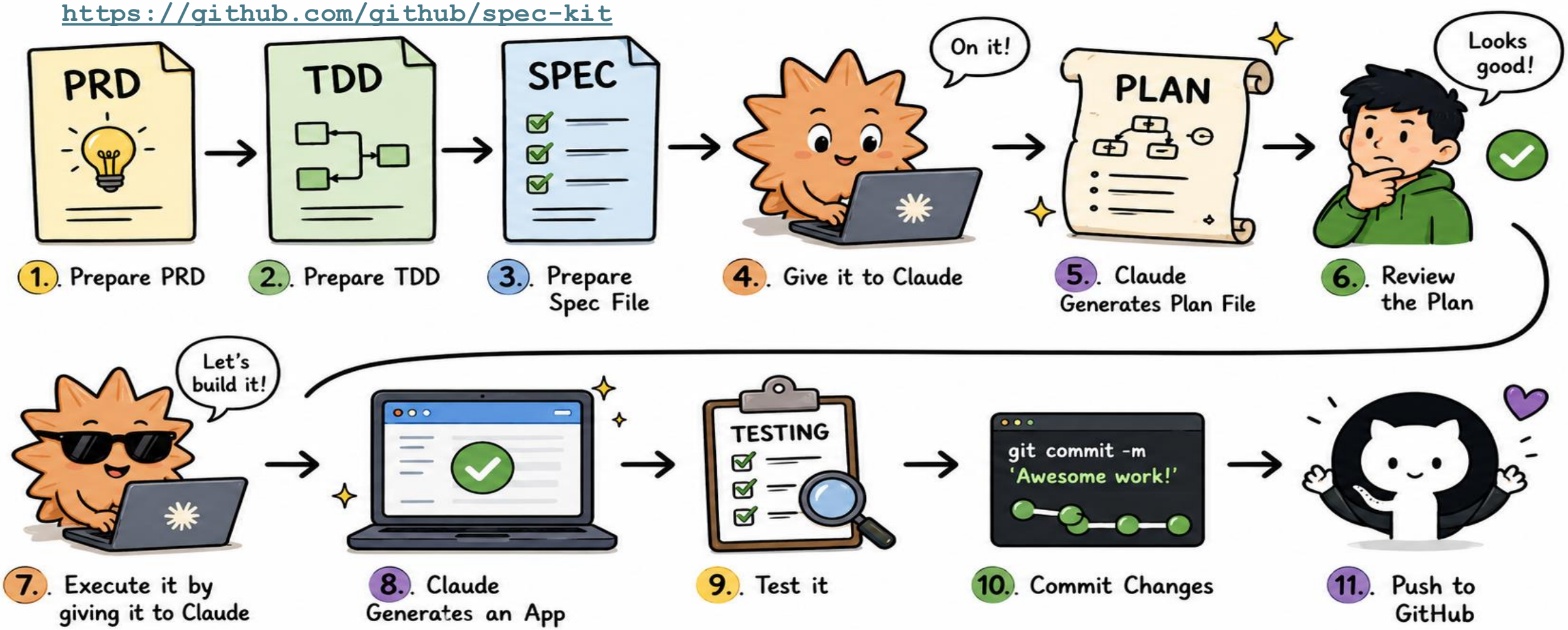
Part – XI: Creating and using Plugins

Part 6
Workflow of Implementing Features & Fixing Bugs

Instructor: Muhammad Arif Butt, PhD

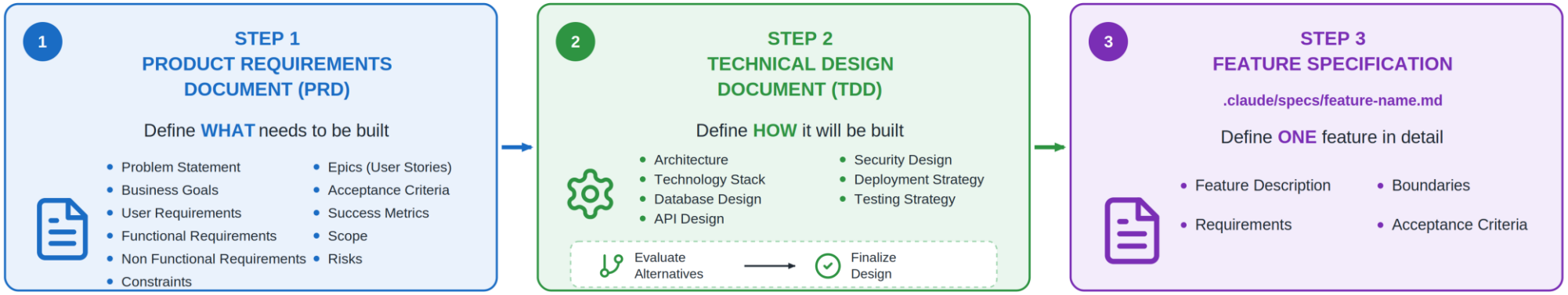
Spec-Driven Development with Claude Code

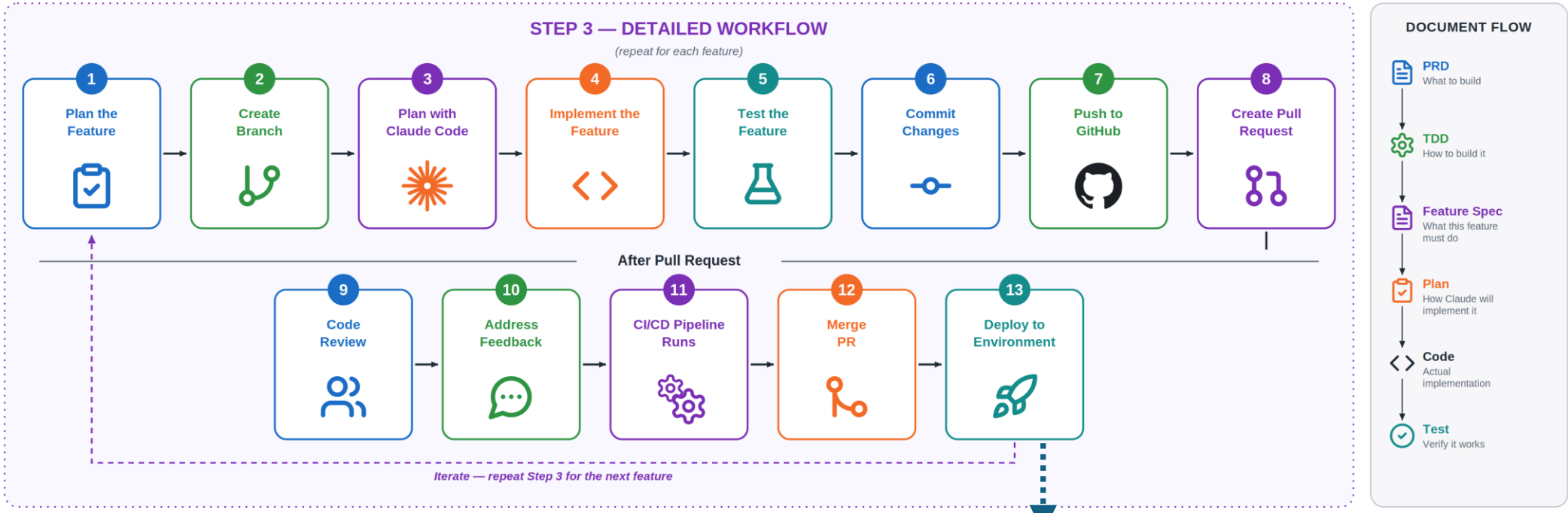
https://github.com/github/spec-kit



Plan it. Build it. Test it. Ship it. Repeat!

Spec-Driven Development with Claude Code





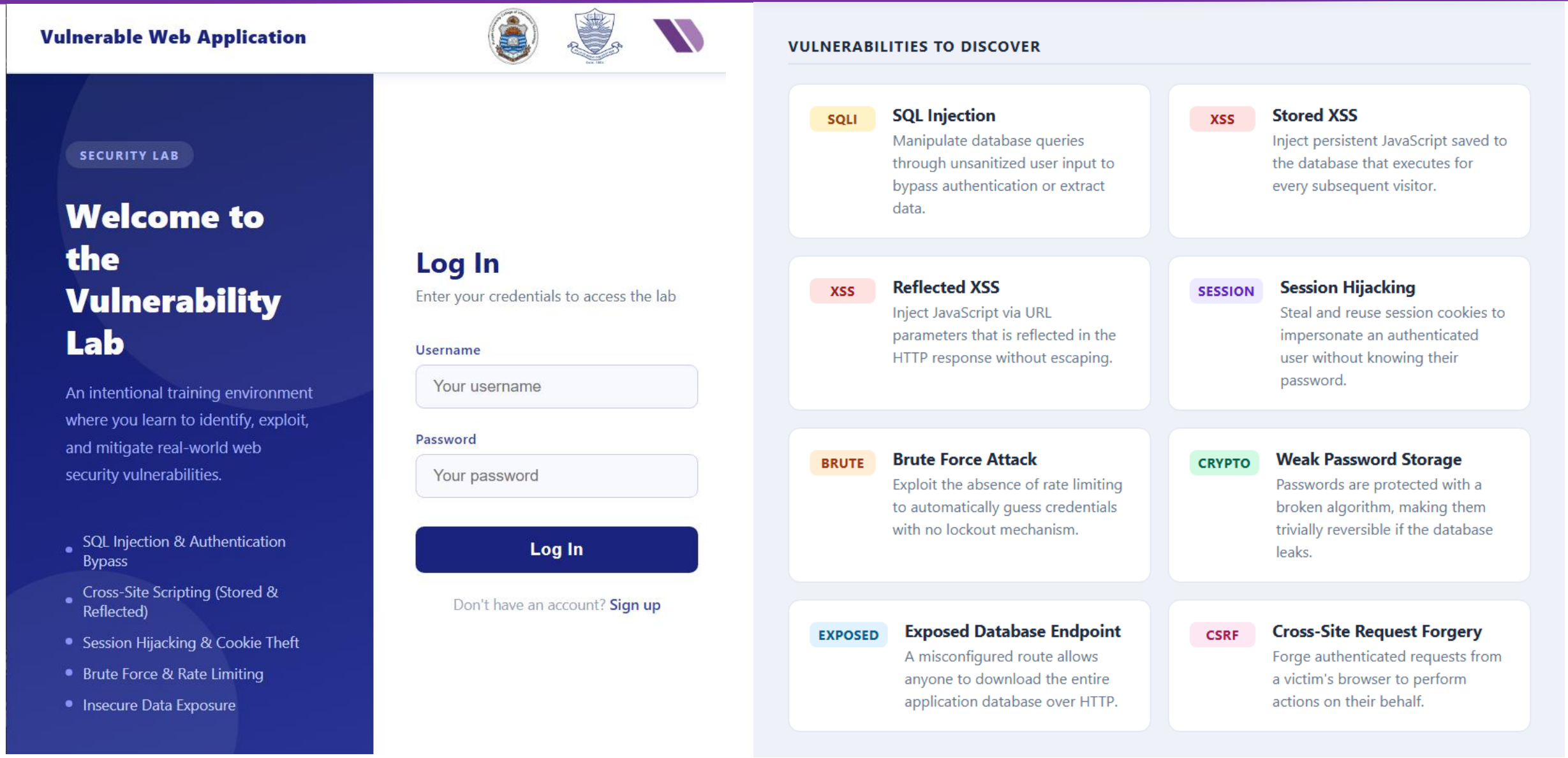


https://arifpucit.github.io/cc-workflow/



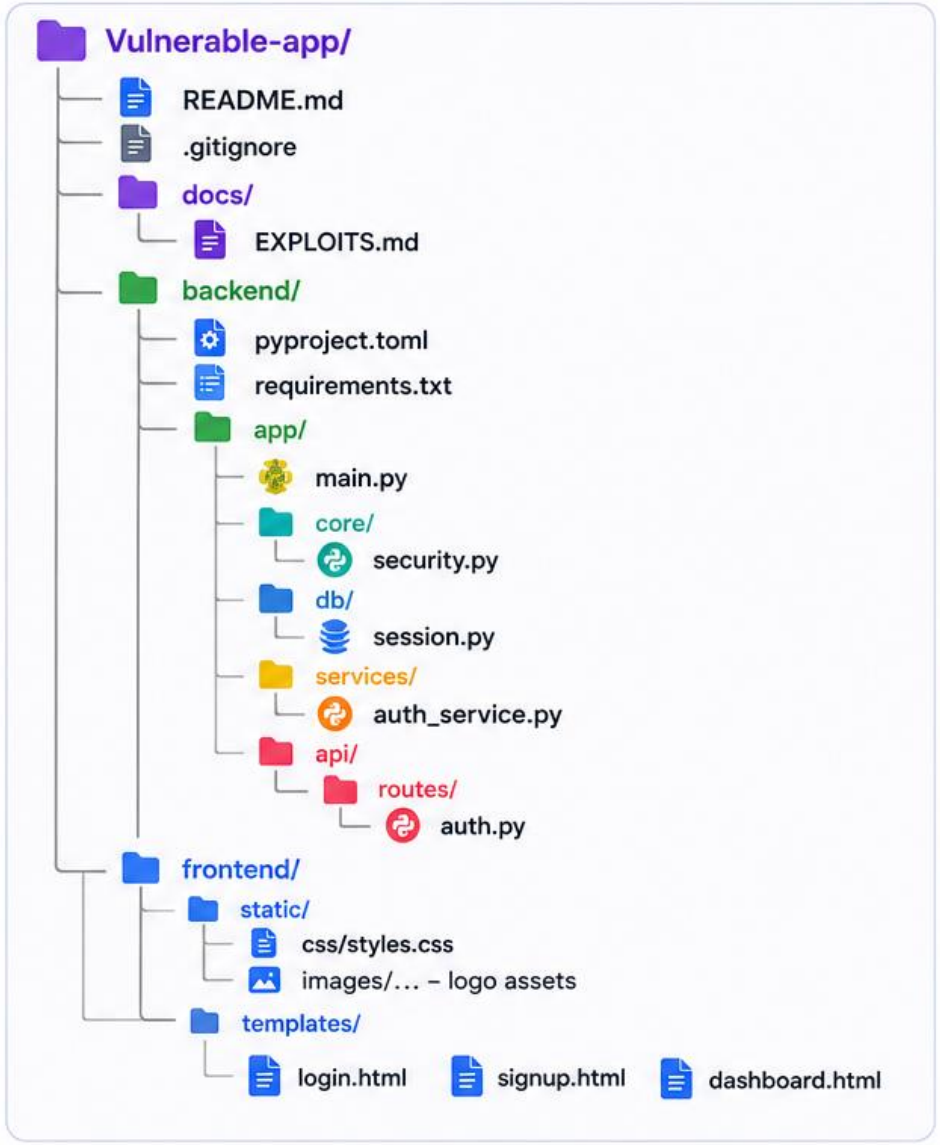
vuln-web-app (v0.1.0)

Vulnerable Web App Interface



https://vuln-web-app.onrender.com/signup

Directory Structure



README.md – overview, install/run instructions, list of intentional bugs

docs/EXPLOITS.md – step-by-step exploitation guide for every flaw

backend/ – FastAPI backend (Python)

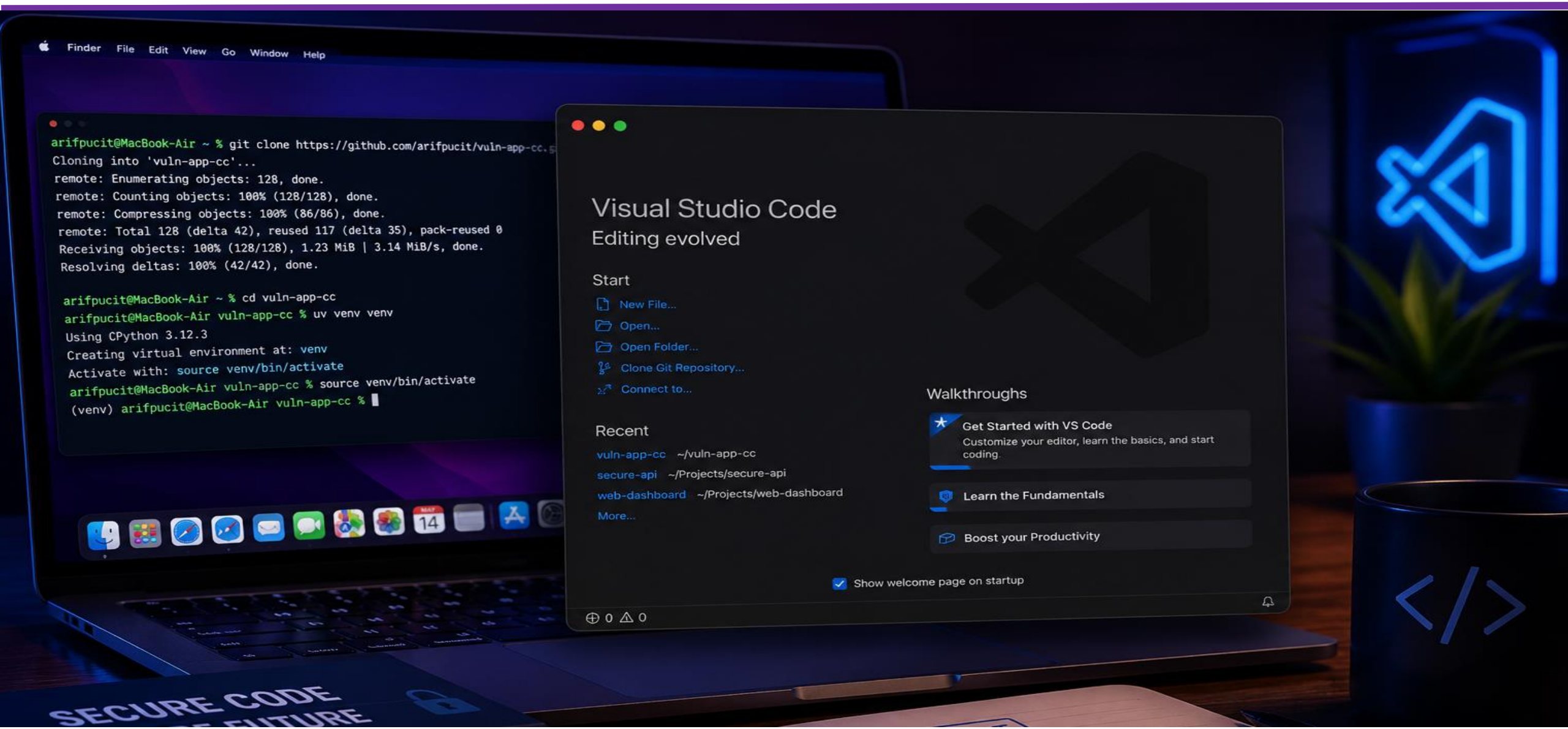
- **main.py** – FastAPI entry point; creates the server and mounts routes
- **core/security.py** – password handling (MD5 hash) and session-secret constant
- **db/session.py** – SQLite connection, creates users table, loads the DB file
- **services/auth_service.py** – business logic for signup / login, queries DB directly
- **api/routes/auth.py** – HTTP endpoints: /signup, /login, /download-db

frontend/ – Web UI

- **static/** – CSS and images (logo assets, etc.)
- **templates/** – HTML pages for the application (login, signup, dashboard)

https://github.com/arifpucit/vuln-web-app

Hands on Practice: Clone the `vuln-web-app`



Adding New Feature (Dark Mode Toggle)

```
SPEC      →  run the task's *spec* prompt       →  produces .claude/specs/<name>.md
PLAN      →  run the task's *plan* prompt        →  produces .claude/specs/<name>-plan.md
EXECUTION →  run the task's *execution* prompt   →  edits the code
```

Adding Dark Mode Toggle

1 Create feature specification
.claude/specs/dark-mode.md

dark-mode.md

```
# Dark Mode Feature

## Overview
Allow users to toggle between
light and dark themes.

## Requirements
- Toggle button in header
- Persist preference
- System preference detection

## Acceptance Criteria
- [x] User can toggle between light and dark mode
- [x] Preference is persisted
- [x] Respects system preference on first visit
- [x] All pages update correctly

## Out of Scope
- Custom themes
- Per-page themes
```

2 Generate plan with Claude Code
.claude/specs/dark-mode-plan.md

dark-mode-plan.md

```
# Dark Mode Implementation Plan

## 1. Analysis
- Review current theme system
- Identify global styles and variables
- Check component dependencies

## 2. Implementation Steps
1. Add theme context and provider
2. Create toggle component
3. Add theme variables (CSS)
4. Implement system preference detection
5. Persist preference in localStorage
6. Update components to use theme variables
7. Test all pages

## 3. Files to Modify
- src/context/ThemeContext.tsx
- src/components/ThemeToggle.tsx
- src/styles/theme.css
- src/App.tsx
- tailwind.config.js

## 4. Testing
- Unit tests for context and toggle
- E2E tests for theme switching

## 5. Acceptance Criteria Mapping
(See dark-mode.md)
```

3 Review & approve plan

Plan Summary

The plan looks comprehensive and coved covers all requirements.

- Analysis
- Implementation Steps
- Files to Modify
- Testing Strategy
- Acceptance Criteria Mapping

Plan Approved
Proceed with implementation

4 Ask Claude Code to implement

Terminal

```
$ claude

You: Implement the dark mode feature as per the
approved plan. Write code, run tests and report back.

Claude Code: Understood. I'll implement the
dark mode feature now.

✓ Analyzing codebase...
✓ Implementing theme context...
✓ Adding toggle component...
✓ Updating styles and components...
✓ Writing tests...
✓ Running tests...

All tests passed!

Dark mode feature implemented successfully.

Summary of changes:
  - 5 files modified
  - 2 files added
  - 18 tests passed
Ready for your review.
```

CLAUDE CODE IMPLEMENTS THE FEATURE

5 Files updated / created

```
src
  components
    ThemeToggle.tsx        A
  context
    ThemeContext.tsx       A
  styles
    theme.css              M
    App.tsx                M
    tailwind.config.js     M
  tests
    theme.test.tsx         A

A Added    M Modified
```

6 Tests run & passed

Terminal

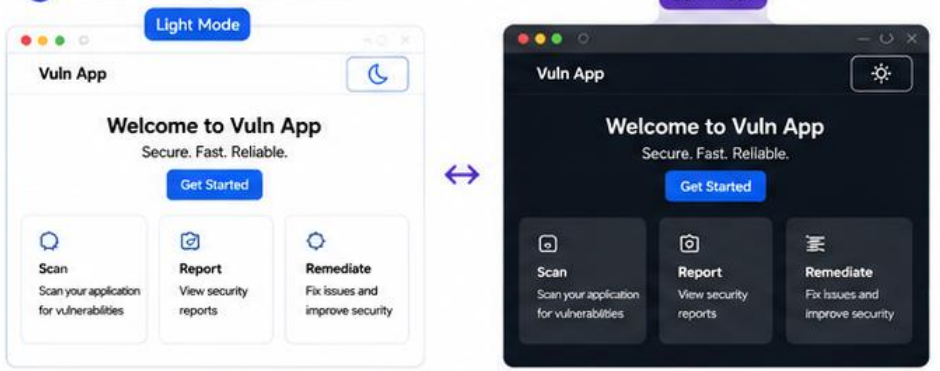
```
$ npm test

PASS  src/tests/theme.test.tsx
PASS  src/tests/ThemeToggle.test.tsx

Test Suites:  2 passed, 2 total
Tests:        18 passed, 18 total
Snapshots:    0 total
Time:         1.23 s

✨ All tests passed!
```

7 Feature working in the website



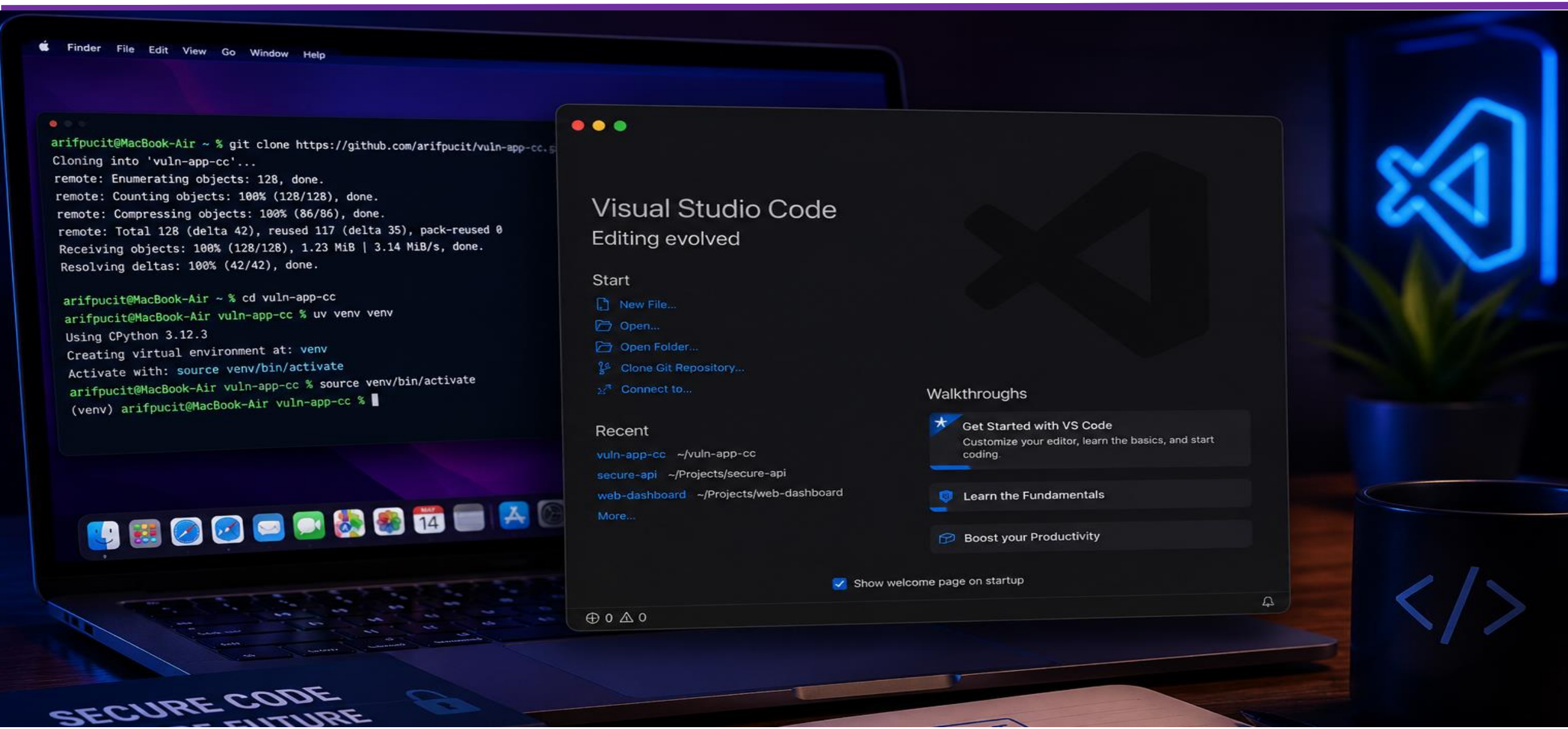
8 Commit & push

Terminal

```
$ git add .
$ git commit -m "feat: add dark mode feature"
[main abc1234] feat: add dark mode feature
7 files changed, 320 insertions(+)
 create mode 100644 src/components/ThemeToggle.tsx
 create mode 100644 src/context/ThemeContext.tsx
 create mode 100644 src/context/ThemeContext.test.tsx
$ git push origin main
Enumerating objects: 15, done.
Counting objects: 100% (15/15), done.
Compressing objects: 100% (7/7), done.
Writing objects: 100% (10/10), done.
☑ Pushed to origin/main
```

✓ Dark Mode feature added successfully and deployed! 🎉

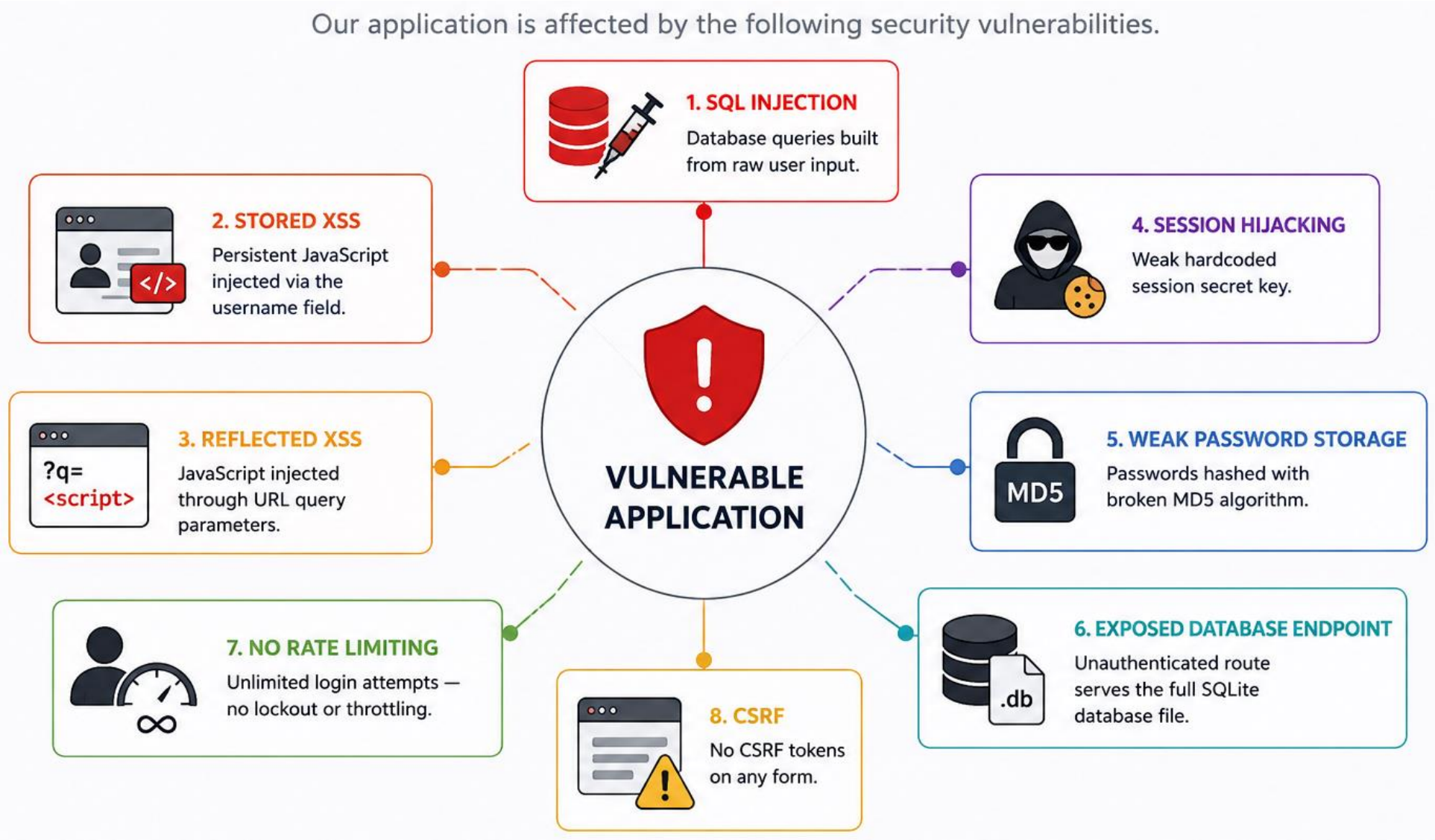
Hands on Practice: Dark Mode Toggle



Fixing Bug
(Weak Hashing: MD5)

```
SPEC      →  run the task's *spec* prompt        →  produces .claude/specs/<name>.md
PLAN      →  run the task's *plan* prompt         →  produces .claude/specs/<name>-plan.md
EXECUTION →  run the task's *execution* prompt    →  edits the code
```

Vulnerabilities



Ask Claude to Detect Bugs and Remediate

1 Provide Code / Context

Developer shares code, logs, scan results, or error messages.

Example Input

```
app.get('/user', (req,res) => {
  const user = db.query("SELECT
  * FROM users WHERE id = " +
  req.query.id);
  res.json(user);
});
```

Possible issue: SQL Injection

2 Claude Analyzes Intelligently

Claude reads and understands the code, context, dependencies and patterns.



- ✓ Understands intent
- ✓ Detects security issues
- ✓ Explains the root cause

3 Vulnerability Detected

Claude pinpoints the issue and explains why it's vulnerable.

Detected Issue

SQL Injection

User input is directly concatenated into the SQL query without validation or parameterization.

Risk

High – Attacker can manipulate the query and access/modify sensitive data.

4 Remediation Suggested

Claude provides a secure fix with explanation.

Suggested Secure Fix

```
app.get('/user', (req,res) => {
  const id = req.query.id;
  const user = db.query(
    "SELECT * FROM users
    WHERE id = ?", [id]
  );
  res.json(user);
});
```

Use parameterized queries (prepared statements) to prevent SQL Injection.

5 Apply & Validate the Fix

Developer applies the fix and re-runs scans/tests.



- ✓ Code compiles
- ✓ Tests pass
- ✓ Vulnerability resolved

6 Continuous Improvement

Claude helps harden the code and prevents similar issues in the future.



- ✓ Secure coding suggestions
- ✓ Best practices
- ✓ Refactor & optimize

HOW CLAUDE CODE HELPS

Deep Code Understanding
Understands complex codebases, frameworks and data flows.

Smart Vulnerability Detection
Finds issues like Injection, XSS, CSRF, IDOR, Insecure Config, etc.

Actionable Remediation
Provides clear, secure fixes with explanations.

Best Practices Guidance
Recommends secure patterns, libraries and configurations.

Context Awareness
Considers project structure, dependencies and environment.

Faster, Safer Development
Peduces manual effort and helps ship secure code.

EXAMPLE: CLAUDE CODE INTERACTION

You: Analyze this code for security issues.

Claude:
⚠ Issue: SQL Injection
Reason: User input is concatenated into the SQL query.
Impact: Attacker can bypass authentication or extract data.
Fix:

```
const user = db.query("SELECT * FROM users WHERE id = ?", [id]);
```

Tip: Always use parameterized queries and validate input.

You: Apply the fix.

Claude: ✓ Fix applied. Code is now secure.

Would you like me to run tests or check for other issues?

SUPPORTED DETECTION & REMEDIATION AREAS

Web Application Security
SQLi, XSS, CSRF, IDOR, SSRF, Auth Issues

Authentication & Authorization
Broken access control, privilege escalation, session management

Configuration & Secrets
Hardcoded secrets, misconfigurations, insecure headers

DevSecOps & CI/CD
Pipeline misconfigurations, IaC issues, container security

Code Quality & Refactoring
Bugs, anti-patterns, performance and maintainability improvements

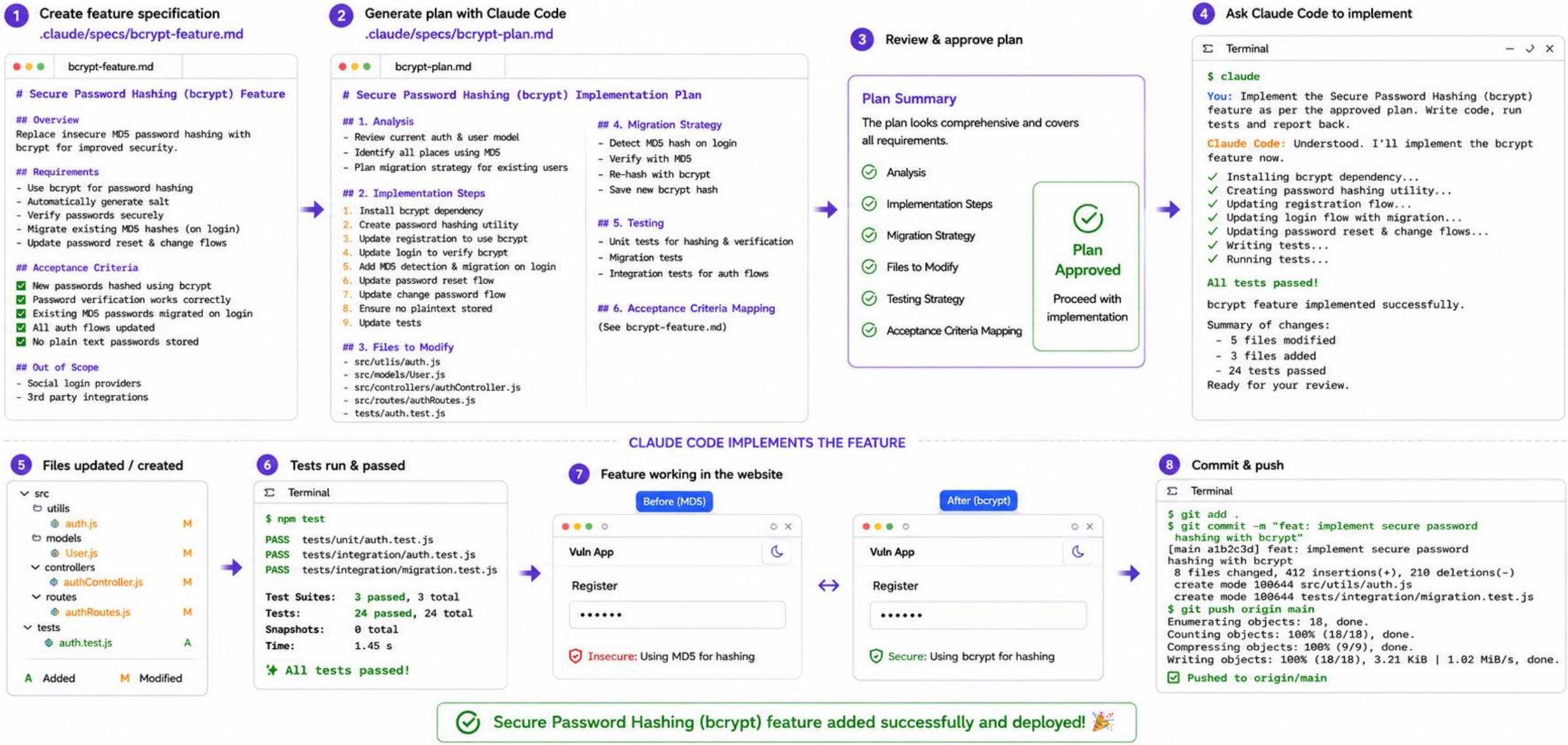
KEY BENEFITS

- **High Accuracy** – Detects real issues with context.
- **Time Saving** – Find & fix issues in minutes.
- **Better Security** – Reduce risk and prevent breaches.
- **Knowledge Sharing** – Learn secure coding as you build.

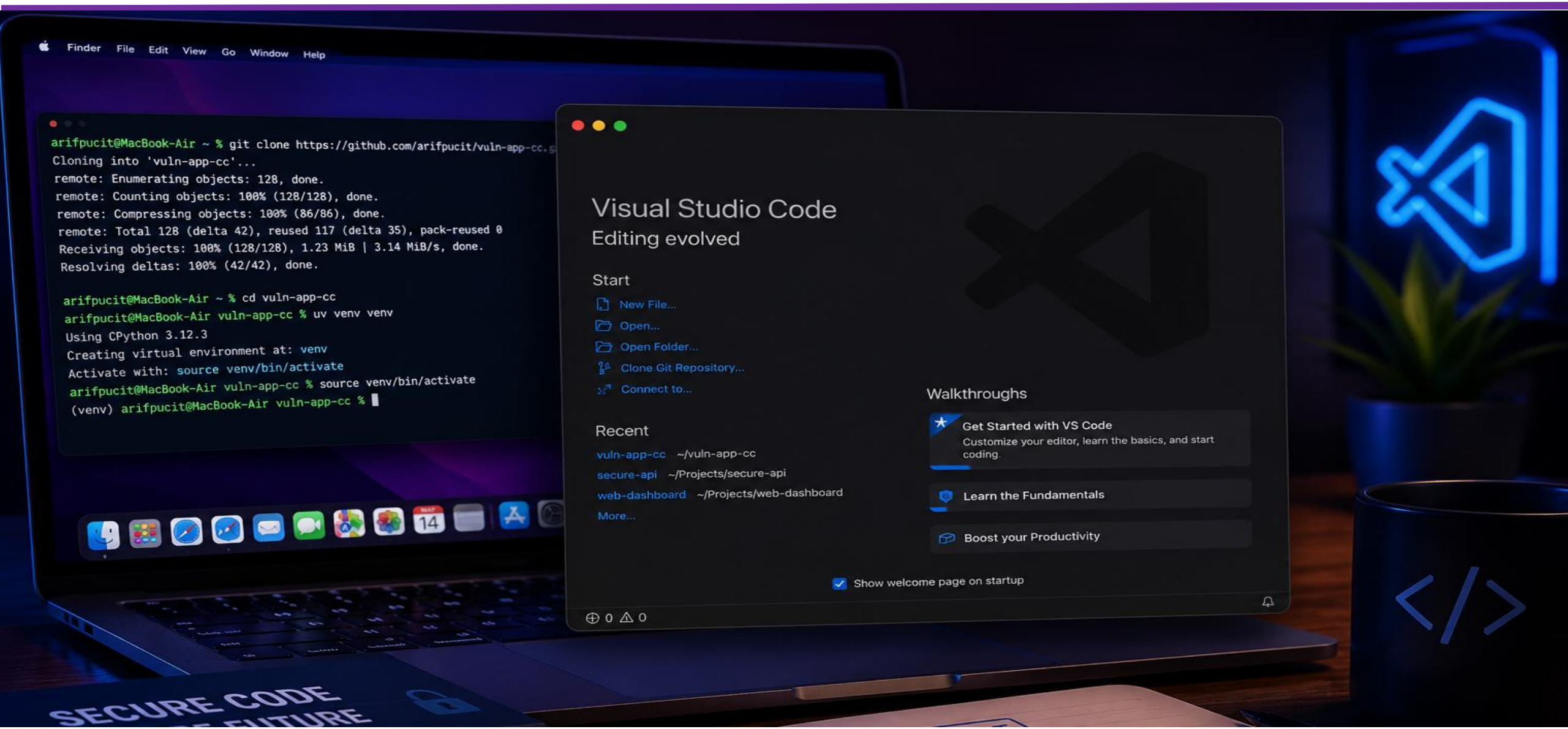
BEST PRACTICES

- Provide clear context and relevant code.
- Review and understand suggested fixes.
- Test thoroughly before deploying changes.
- Keep Claude updated with latest scans/logs.

Weak Hashing (MD5)



Hands on Practice: Weak Hashing (MD5)



Q/A



Thank you!